

Kauffman Bracket Computation: Topology-Based Memoization Cuts Calls by 32.5%, but Lookup Overhead Costs More

Alfie James Jenkins

Falmouth University – COMP203 Optimization

The Knot Invariant Calculator

C++ program that computes the Jones polynomial of mathematical knots using the Kauffman bracket.

Knots are closed curves in 3D space. The Jones polynomial distinguishes knot types and shows up in molecular biology, quantum physics, and polymer science. Computing it via the Kauffman bracket is $O(2^n)$ — a 21-crossing knot needs 4.2 million recursive calls.

Input: Dowker–Thistlethwaite (DT) notation from databases like KnotInfo.

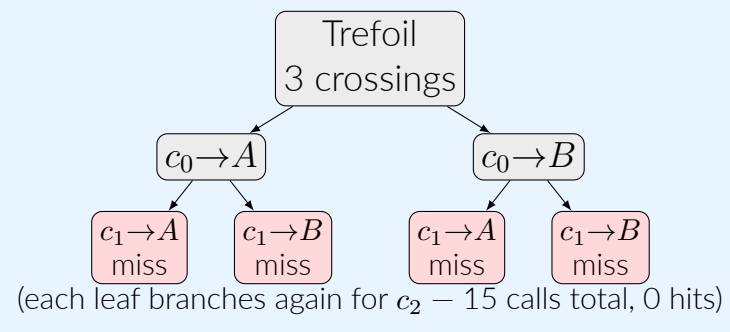
Output: Jones polynomial $V_K(A)$, verified against published tables.

The naive approach: history-based caching

Initial implementation

Cache key: record which smoothings have been applied, e.g. `DiagramState{0→A, 1→B, 2→A}`.

Crossings are always processed in ascending order (0, 1, 2, ...), creating a perfect binary tree where every root-to-leaf path is unique. No two branches ever reach the same state.



Knot	n	Calls	Hits	Rate
Trefoil	3	15	0	0%
Figure-8	4	31	0	0%
T(2,7)	7	255	0	0%

Every `cache.find()` returned `end()`. The cache stored entries and never used a single one. The reason, in hindsight: deterministic traversal means every path is unique, so no two branches ever share a state to cache.

Key algorithm features

The Kauffman bracket expansion

Each crossing is smoothed in two ways, creating a recursive binary tree:

$$\langle K \rangle = A \cdot \langle \text{A-smooth} \rangle + A^{-1} \cdot \langle \text{B-smooth} \rangle$$

$$\text{crossing} \quad \text{A-smooth} \quad \text{B-smooth}$$

Base case: When all crossings are smoothed, count resulting circles k :

$$\langle k \text{ circles} \rangle = (-A^2 - A^{-2})^{k-1}$$

Jones polynomial

Normalise the bracket by the writhe w :

$$V_K(A) = (-A)^{-3w} \cdot \langle K \rangle$$

Arc endpoint model

A knot with n crossings has $2n$ arc segments. Smoothing reconnects endpoints according to crossing orientation. Union-Find with path compression tracks connected components in $O(\alpha(n))$ amortised time.

Implementation

Language & containers: C++17. `std::map<int,int>` stores sparse Laurent polynomials (power $\rightarrow$ coefficient); zero-coefficient terms are erased after each addition.

Input parsing: Each DT notation integer maps to one crossing. Sign encodes orientation (+1 or -1); absolute value gives the paired arc position in $[0, 2n)$.

Timing: `std::chrono::high_resolution_clock` measures bracket computation alone, separate from Jones normalisation. Cached and uncached runs use fresh `KauffmanBracket` instances so neither warms the other's state.

Build: `g++ -std=c++17 -O2` via Makefile. Git history tracks the path from an early heuristic prototype to the topology-based cache.

References

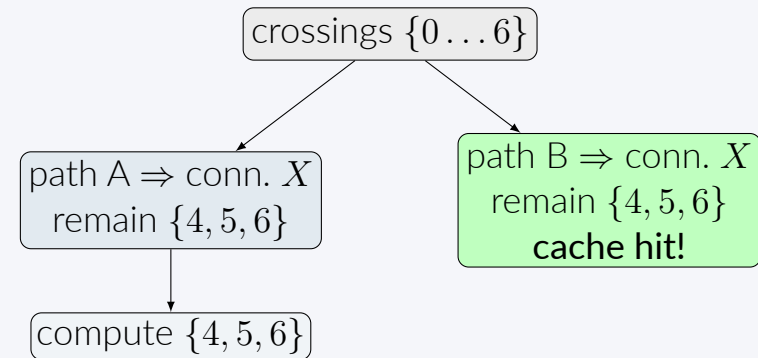
- Jones, V. F. R. (1985). *A polynomial invariant for knots via von Neumann algebras*. Bull. Amer. Math. Soc., 12(1), 103–111.
- Kauffman, L. H. (1987). *State models and the Jones polynomial*. Topology, 26(3), 395–407.
- Musfeldt, C. (2024). *Development of a Python-Based Software for Calculating the Jones Polynomial*. Arizona State University.
- KnotInfo: <https://knotinfo.math.indiana.edu>

Solution: Topology-Based Caching

The key idea

Stop recording what smoothings were applied. Record what they produced.

Two different A/B choice sequences can reach the same arc connectivity with the same crossings still to process. At that point the remaining computation is identical — the second branch can skip it entirely.



Cache key structure

```
1 struct CacheKey {
2     vector<int> connectivity; // canonical arc IDs
3     int remaining;           // bitmask: bit i = crossing i unsmoothed
4
5     bool operator< (const CacheKey& other) const {
6         if (remaining != other.remaining)
7             return remaining < other.remaining;
8         return connectivity < other.connectivity;
9     }
10 };
11
12 map<CacheKey, Polynomial> cache;
```

The `remaining` bitmask clears bit i via `remaining & ~(1 << i)`. **Safe limit: 30 crossings** with a 32-bit signed `int` (bit 31 shift is undefined behaviour in C++).

Canonical connectivity

The equivalence problem

Two Union-Find states may be structurally identical but differ in internal representation depending on construction order. Raw roots `[7,7,2,2]` and `[3,3,0,0]` encode the same pattern — a canonical form is needed for cache comparisons to work.

Canonicalization algorithm

```
1 vector<int> canonicalize(vector<int> uf, int n) {
2     vector<int> canonical(2 * n);
3     map<int,int> id_map; // root -> canonical ID
4     int next_id = 0;
5
6     for (int i = 0; i < 2 * n; i++) {
7         int root = uf_find(uf, i);
8         if (id_map.find(root) == id_map.end())
9             id_map[root] = next_id++;
10        canonical[i] = id_map[root];
11    }
12    return canonical; // components numbered 0, 1, 2, ...
13 }
```

Endpoints scanned left-to-right, IDs assigned in order of first appearance. Two equivalent topologies always produce the same key regardless of how they were built.

Complexity: $O(2n \cdot \log k)$, k = distinct components (typically $k < 5$).

Smoothing operations

At each crossing, two arc endpoints arrive and two depart. A-smoothing and B-smoothing reconnect them in one of two patterns depending on crossing sign:

```
1 void apply_smoothing(vector<int>& uf,
2     const pair<int,int>& pos, int sign, char type, int n) {
3     auto [p, q] = pos;
4     int before_p = (p - 1 + 2*n) % (2*n);
5     int before_q = (q - 1 + 2*n) % (2*n);
6     bool cross_connect = (type == 'A') == (sign > 0);
7     if (cross_connect) {
8         uf_unite(uf, before_p, q);
9         uf_unite(uf, before_q, p);
10    } else {
11        uf_unite(uf, before_p, before_q);
12        uf_unite(uf, p, q);
13    }
14 }
```

Cross-connect (A-smooth of positive crossing): joins the strand arriving at p to the one leaving at q , and vice versa.

Parallel-connect (B-smooth of positive crossing): joins both arrivals and both departures, potentially merging or splitting components.

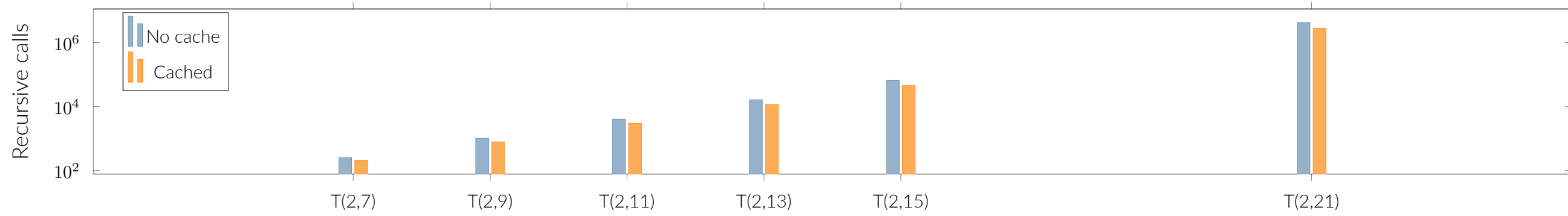
Performance Results

Call count reduction

Knot	n	No cache	Cached	Hit rate	Saved
T(2,7)	7	255	211	10.4%	17.3%
T(2,9)	9	1,023	791	14.7%	22.7%
T(2,11)	11	4,095	3,017	17.9%	26.3%
T(2,13)	13	16,383	11,675	20.2%	28.7%
T(2,15)	15	65,535	45,663	21.8%	30.3%
T(2,21)	21	4,194,303	2,831,623	24.1%	32.5%

For T(2,21), 1.36 million calls are skipped. The cache is finding genuine duplicate subproblems — the lookup cost is just too high to profit from them.

Call reduction (log scale)



Knot	n	No cache (μ s)	Cached (μ s)	Speedup
T(2,7)	7	61	132	0.46×
T(2,11)	11	944	2,232	0.42×
T(2,15)	15	16,947	58,971	0.29×
T(2,21)	21	1,186,334	3,637,637	0.33×

Single-run measurements; variance $\pm 20\%$ between runs.

Fewer calls, but slower. Each lookup requires `canonicalize()` — a linear scan with `std::map` insertions — followed by an $O(n)$ vector comparison in the cache lookup itself. At 20–24% hit rates, that overhead costs more than it saves.

Verified correctness

- Figure-8 (4₁): $A^8 - A^4 + 1 - A^{-4} + A^{-8}$ ✓
- T(2,15): 10-term polynomial, powers spaced by 4 from A^{-19} to A^{17} ✓

When does memoization help?

Torus knots are too regular. Intermediate states rarely coincide, so hit rates top out around 24% — not enough to offset the lookup cost. Pretzel and composite knots have denser crossing interactions; 40–50%+ hit rates seem plausible there, which is where the overhead would start to pay off.

Callgrind: where the time goes

Profiled with `valgrind --tool=callgrind` (debug build, T(2,7)–T(2,13)):

Instrs.	Function	Cause
>20%	<code>_Rb_tree</code> internals	<code>id_map</code> inside <code>canonicalize()</code>
5.4%	<code>lexicographical_compare</code>	cache key vector compare
4.8%	<code>uf_find</code>	path compression
1.2%	<code>malloc/free</code>	temporary map per call

The `std::map<int,int> id_map` inside `canonicalize()` is heap-allocated and destroyed on every recursive call. Red-black tree operations alone exceed 20% of all instructions — directly explaining the 2–3.5× slowdown despite fewer calls.

Two implementation fixes would help regardless of knot family: replace `std::map` with `std::unordered_map` ($O(1)$ average lookup instead of $O(\log n)$), and lazily canonicalize — skip computing the canonical form unless an entry is actually being stored. Array-based canonicalization (replacing `id_map` with a `vector<int>[2n]`) would also eliminate the per-call heap allocation.

Design notes and future work

Sparse polynomial storage: `std::map<int,int>` stores only non-zero terms. T(2,15)'s bracket spans A^{-19} to A^{17} in steps of 4 — 10 terms out of a possible range of 19.

Value-based recursion: Union-Find passed by value so each branch gets an independent copy. Works cleanly up to around $n = 25$; beyond that the copy overhead probably dominates and a different strategy is needed.

Still to do: profile with Valgrind to confirm where the overhead actually lives, test on pretzel and composite knots, implement the hash map, port to Raspberry Pi for an embedded comparison.